Blood Cell Classification with ResNet-18:

From Pretrained Features to Fine-Tuning

Term Project Proposal | 2102666 Fundamentals of Deep Learning and Applications

1. Team members and route

Member 1: Kaung Min Khant | Student ID: 6872011721

Member 2: Padamyar Khin | Student ID: 6872054721

Route A – Track 1: Image Classification by Transfer Learning

2. Problem

We will classify color microscope images into eight blood-cell categories using BloodMNIST. This educational study will examine whether pretrained visual features transfer effectively to blood-cell images when labelled training data are limited. We will compare a frozen linear probe, full fine-tuning and training from scratch using the same ResNet-18 architecture. Incorrect predictions may confuse similar cell categories, so we will examine errors for each class rather than rely on accuracy alone. The model is intended for research and classroom demonstration, not clinical diagnosis.

3. Data

We will use BloodMNIST from MedMNIST [1, 2], derived from the peripheral blood-cell microscope dataset of Acevedo et al. [3]. The source contains individual cell images from people without the reported infectious, haematological or oncological conditions; its collection setting limits generalisation to other populations and laboratories. BloodMNIST contains **17,092 RGB images in eight classes**; we will use the **224 x 224** release under **CC BY 4.0** and cite both sources. The labels are basophil, eosinophil, erythroblast, immature granulocytes, lymphocyte, monocyte, neutrophil and platelet.

Official split	Training	Validation	Test	Total
Images	11,959	1,712	3,421	17,092

Access was confirmed by loading the training and validation data in Colab on 17 October 2026 and displaying labelled training samples. We will retain the official splits, inspect training class counts, and document duplicate or subject-overlap concerns if metadata permit. The test set will remain reserved.

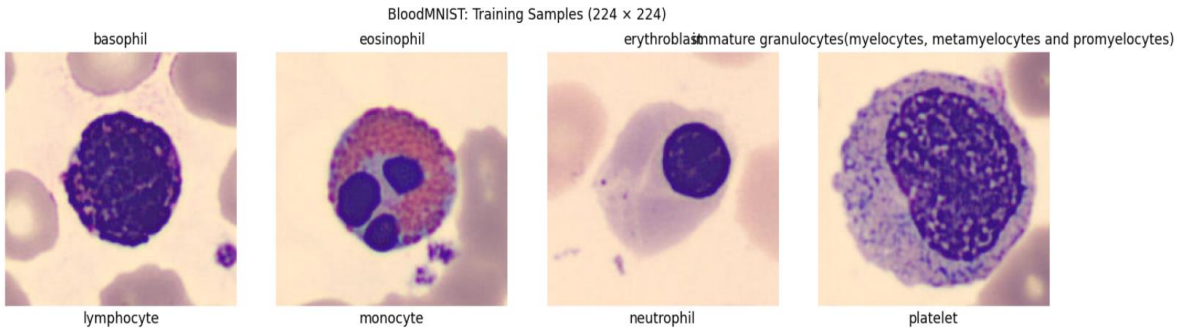


Figure 1. Training samples displayed in the team's Colab

4. Baselines

Trivial: always predict the most frequent training class. **Simple:** freeze an ImageNet-pretrained ResNet-18 backbone and train only an eight-class linear head (Laboratory 7, Part 2). We will obtain both baseline results before interpreting the full-model results.

5.Model

Following the course CNN and transfer-learning material and Laboratory 7, we will replace ResNet-18's classifier with an eight-output linear layer. Cross-entropy matches the mutually exclusive labels. We propose AdamW with weight decay 10^{-4} for adaptive updates and regularization, a cosine learning-rate schedule, batch size 32 and a common 20-epoch budget. We will apply ImageNet normalization consistently. The linear probe keeps the backbone, including batch-normalization statistics, fixed; full fine-tuning updates all weights; the scratch model starts from random weights.

6.Evaluation

Primary metric: macro-F1. We will also report per-class precision and recall, macro one-vs-rest ROC-AUC from class probabilities, and a confusion matrix. Every metric table will include the evaluated split's class counts and prevalence. For every configuration in the main table, we will use seeds **42, 123 and 2026** and report mean and sample standard deviation, never just the best run. Validation macro-F1 will determine learning rates, checkpoints and the preferred strategy. After freezing all decisions, we will evaluate the prespecified models and seeds in one final test-evaluation stage; no test-driven retuning will follow.

7.The comparison

Experiment A: vary only the training strategy: frozen linear probe, full fine-tuning, or random initialization. All arms use the same splits, preprocessing, batch size and 20-epoch training budget, initially without augmentation. Each arm receives three learning-rate trials: {0.0001, 0.001, 0.01} for the probe and scratch model, and {0.00001, 0.0001, 0.001} for fine-tuning. Each arm is tuned independently on validation data; settings are then rerun with three seeds for the main comparison.

Experiment B: select Experiment A's best strategy by validation macro-F1 and compare it with and without training-only augmentation (horizontal/vertical flips and rotations up to 15 degrees). Keep other settings fixed and give each arm its own equal-size learning-rate sweep. Validation and test images receive no random augmentation. We expect pretraining to improve performance or convergence, and augmentation may reduce overfitting; these are hypotheses, not established results.

8. Plan

Weeks	**Planned work**
19 October	Finalize data audit, counts, preprocessing and fixed splits.
26 October	Run majority baseline and frozen linear probe; profile runtime.
2 November	Run fine-tuning and scratch models; tune learning rates.
9 November	Prepare first results and curves; draft progress report.
16 November	Submit progress report; complete strategy comparisons.
23 November	Complete augmentation and three-seed runs; analyze validation errors.
30 November	Freeze choices; final test evaluation; report, reproducible code and demo.
7 December	Present and demonstrate; submit final report and repository.

9. Risks and fallback

GPU limits or memory errors may interrupt runs: we will save checkpoints, use mixed precision where appropriate, and reduce the common batch size if needed. If profiling shows a run could exceed two hours, we will reduce the epoch budget equally across comparison arms before the main experiments and document the change. Overfitting or poor minority-class recall will be analyzed using validation curves and per-class metrics; no improvement will be claimed without evidence. Missing patient identifiers will be reported as a limitation of leakage verification.

10. Compute

We will use Python, PyTorch/torchvision and a free Colab T4 GPU. Preliminary planning estimates are 0.25-0.75 GPU-hours per linear-probe run and 0.5-1.5 GPU-hours per full fine-tuning or scratch run; these are unmeasured estimates to verify by timing pilot epochs. Every reported run must finish in under two hours on a T4. GitHub repository will contain the environment, configurations, seeds, data instructions and one notebook reproducing the headline result. We will declare ChatGPT assistance with dataset-inspection code, plus any later AI-tool use.

References

[1] Yang et al. MedMNIST v2 - A large-scale lightweight benchmark for 2D and 3D biomedical image classification. Scientific Data 10, 41 (2023). doi:10.1038/s41597-022-01721-8.
[2] MedMNIST official dataset information: medmnist.com; BloodMNIST metadata: MedMNIST/medmnist/info.py.
[3] Acevedo et al. A dataset of microscopic peripheral blood cell images for development of automatic recognition systems. Data in Brief 30, 105474 (2020).